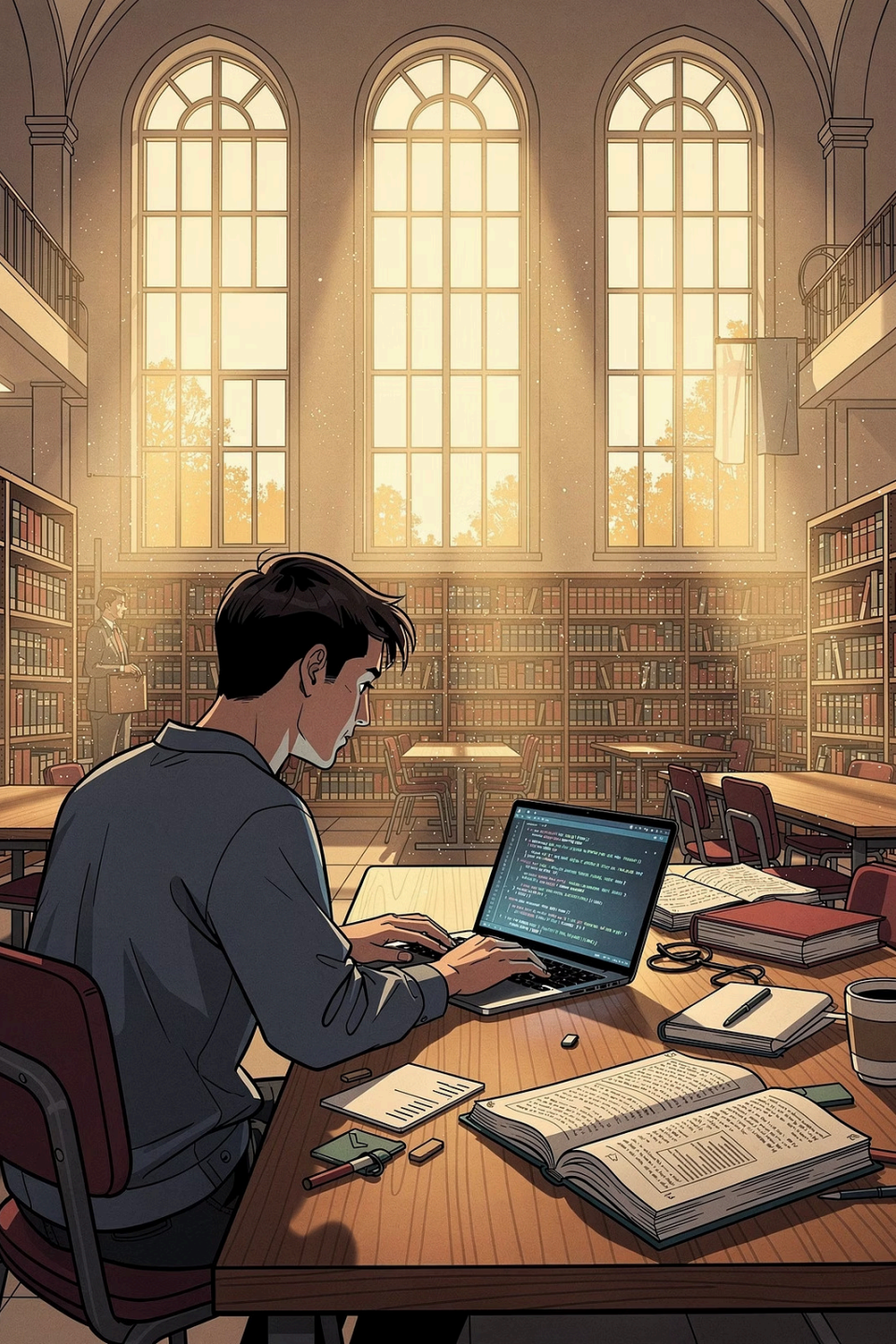




Chapter 7: While Loops

040223101 Computer Programming for Mathematics 1 – Department of Mathematics, Faculty of Applied Science, King Mongkut's University of Technology North Bangkok

Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul | Semester 1, Academic Year 2026

Learning Objectives

Upon completing this chapter, students will be able to:

O1

Compare Loop Types

Explain the differences between `while` loops and `for` loops and identify when each is appropriate.

O2

Write while Statements

Write `while` commands with correctly defined stopping conditions to control loop execution.

O3

Avoid Infinite Loops

Recognize and prevent infinite loop situations by ensuring loop variables change within the loop body.

O4

Use `break` and `continue`

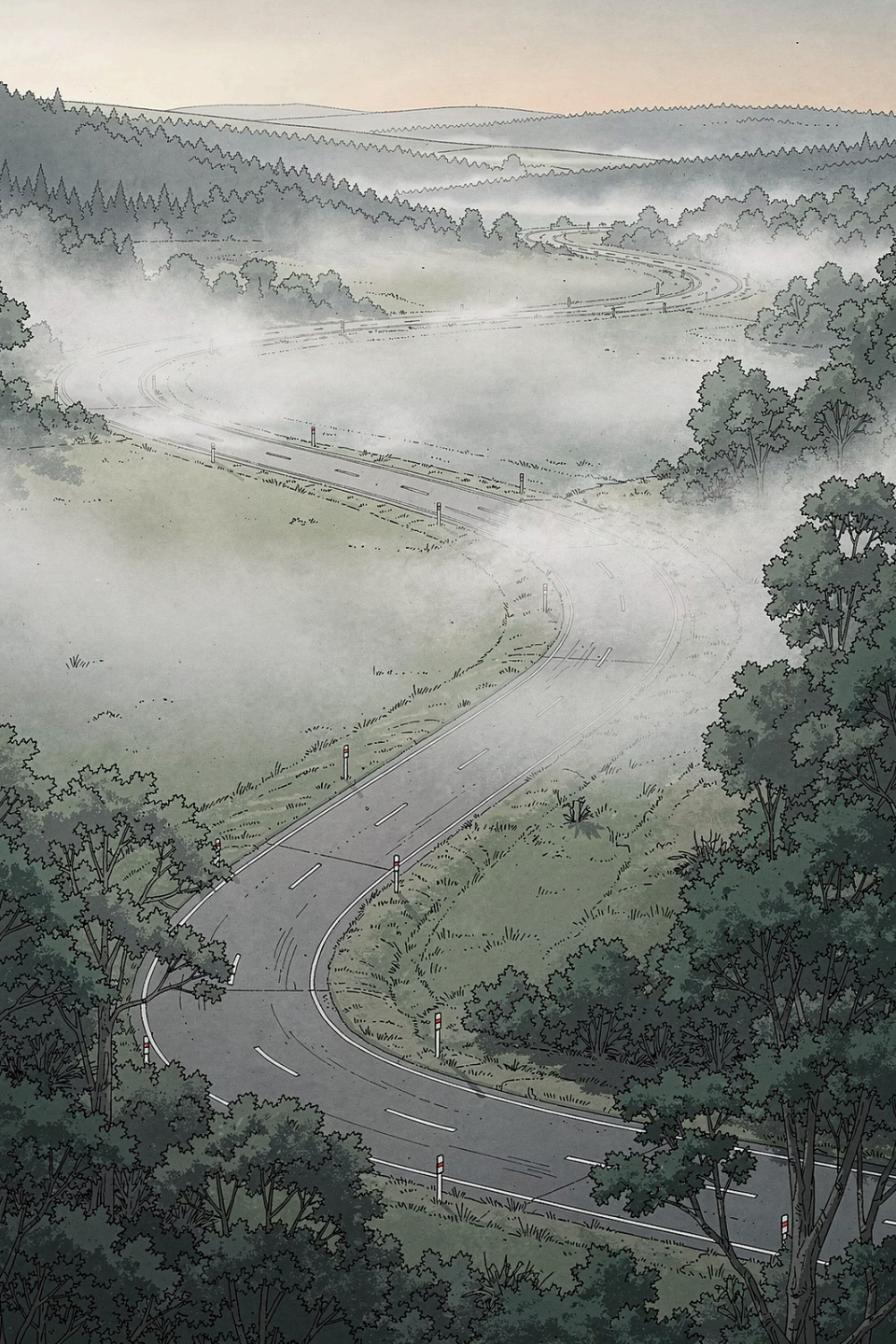
Apply `break` and `continue` statements to control loop flow with precision.

O5

Apply to Numerical Problems

Use `while` loops to solve numerical problems such as convergence calculations and input validation.

 This chapter aligns with Course Learning Outcome **CLO 3**.



Chapter 7: Conditional Repetition Structures (While Loops)

A **for** loop is ideal when the number of iterations is known in advance. However, there are many situations where we do not know how many repetitions will be needed – for example, repeatedly accepting input until the user provides a valid value, or computing until a result converges. In these cases, the **while loop** is the right tool: it repeats *as long as a condition remains true*.

7.1 The while Statement

How It Works

The `while` loop checks its condition **before every iteration**. If the condition is true, the block of statements executes; then the condition is checked again. This repeats until the condition becomes false, at which point execution exits the loop.

The critical requirement: there must be a statement inside the loop that eventually makes the condition false – otherwise the loop runs forever.

Syntax Pattern

```
while <condition>:  
    <statement(s)>  
    <update variable>
```

Key Rule

Always ensure the variable used in the condition is **modified inside the loop body** so the condition can eventually become false.

7.1 First while Loop Example

File: 07while1.py – This example counts from 1 to 5, printing the round number each iteration, then exits.

Source Code

```
i = 1

while i <= 5:
    print("Round", i)
    i += 1

print("Exit loop")
```

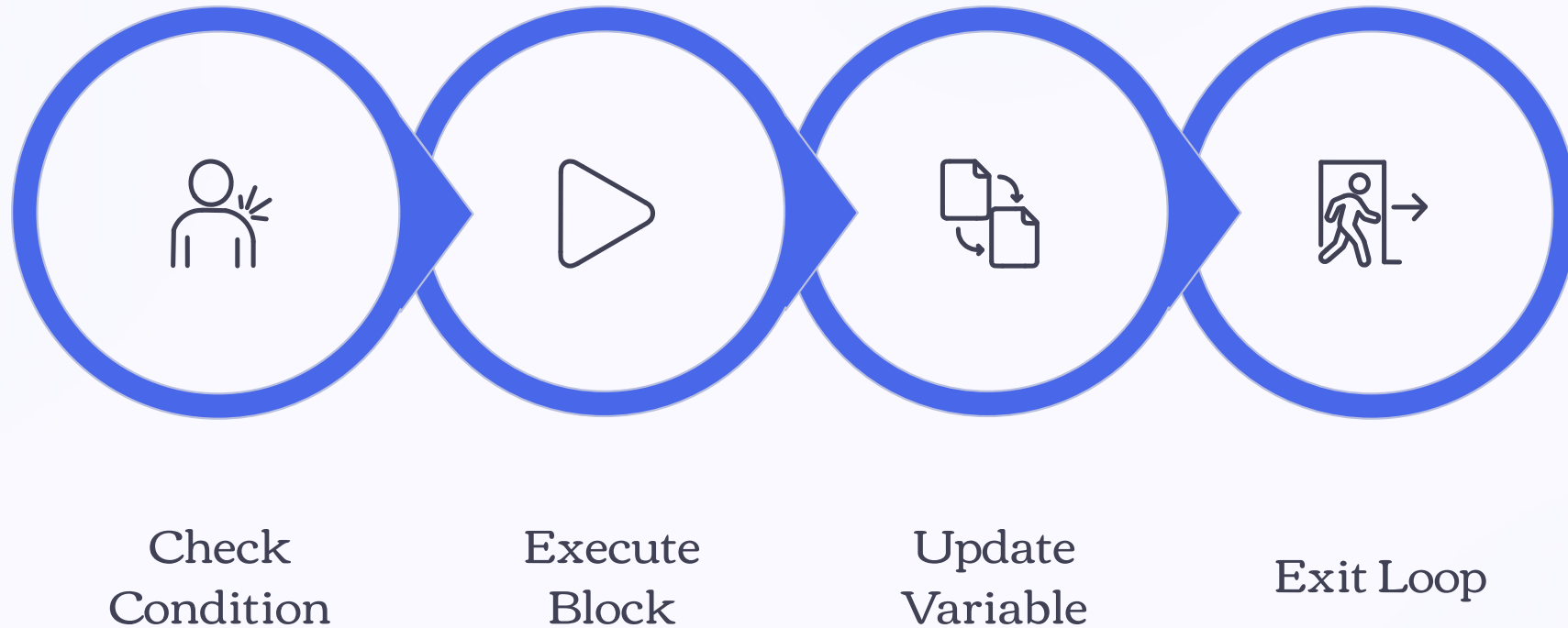
Output

```
Round 1
Round 2
Round 3
Round 4
Round 5
Exit loop
```

Trace

The variable `i` starts at 1. Each iteration prints the round and increments `i` by 1. When `i` reaches 6, the condition `i <= 5` becomes false and the loop exits, printing "Exit loop".

7.1 Visualizing the while Loop Flow



The loop always begins by evaluating the condition. Only if it is **true** does the body execute. After the body runs, control returns to the condition check. When the condition is **false**, execution jumps past the loop entirely. This pre-check behavior distinguishes `while` from a `do-while` structure (not available in Python).

7.2 Infinite Loops and How to Avoid Them

If the condition is always true and no statement inside the loop changes it to false, the loop runs forever – this is called an **infinite loop**, one of the most common programming errors.



7.2 Infinite Loop: The Classic Mistake

Incorrect Code

```
# Example of a bug — forgot to increment i
i = 1
while i <= 5:
    print(i)
    # No i += 1 → infinite loop!
```

Because `i` is never incremented, it stays equal to 1 forever. The condition `i <= 5` is always true, so the program prints 1 endlessly without stopping.

How to Fix It

```
i = 1
while i <= 5:
    print(i)
    i += 1 # ← This line is essential
```

Emergency Stop

If your program freezes due to an infinite loop, press **Ctrl + C** in the terminal window to force-stop execution.

Prevention Checklist

- Identify which variable controls the condition
- Confirm that variable changes inside the loop
- Trace through the first few iterations manually
- Consider edge cases where the condition might never become false

7.3 The break and continue Statements

break

Exits the loop **immediately**, even if the condition is still true. Execution jumps to the first statement after the loop block. Useful when a target value is found and further iteration is unnecessary.

continue

Skips the **remaining statements** in the current iteration and jumps back to the condition check for the next round. Useful for filtering out unwanted values without exiting the loop entirely.

- ❏ Both `break` and `continue` work inside both `while` and `for` loops. They give you fine-grained control over loop execution beyond what the condition alone can express.

7.3 Using break – Stop When Found

File: 07break.py – Find the smallest positive integer whose square exceeds 100.

Source Code

```
# break: stop when target condition is met
n = 1
while True:
    if n * n > 100:
        break
    n += 1

print(f"Smallest positive integer whose
      square exceeds 100 is {n}")
```

Output

Smallest positive integer whose
square exceeds 100 is 11

Explanation

The loop uses `while True` – an intentionally infinite loop – relying entirely on `break` to exit. When `n = 11`, the condition `11 × 11 = 121 > 100` is satisfied, so `break` fires and the loop ends. This pattern is common when the exit condition is complex or checked mid-loop.

7.3 Using continue – Skip Unwanted Values

File: 07continue.py – Print only odd numbers from 1 to 9 by skipping even numbers.

Source Code

```
# continue: skip even numbers,  
# print only odd numbers  
i = 0  
while i < 10:  
    i += 1  
    if i % 2 == 0:  
        continue  
    print(i, end=" ")
```

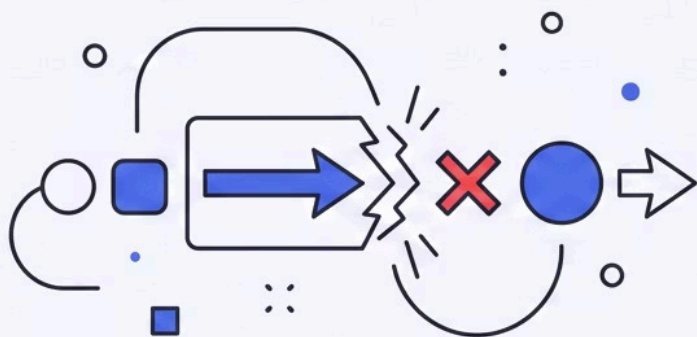
Output

```
1 3 5 7 9
```

Explanation

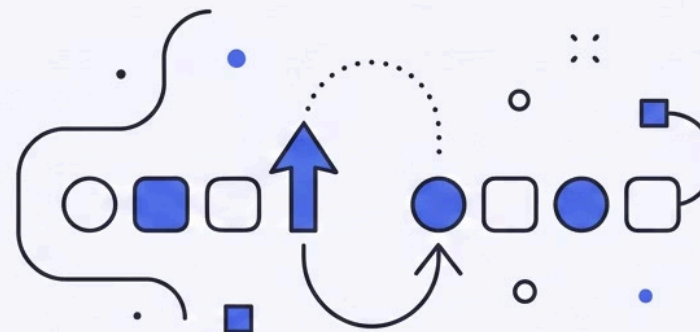
Each iteration increments `i` first, then checks if it is even. If `i % 2 == 0` (even), `continue` skips the `print` statement and returns to the condition check. Only odd values reach the `print` line. Note that `i += 1` is placed **before** the `continue` check – if it were after, even numbers would cause an infinite loop.

break vs. continue – Side-by-Side



BREAK

**exits the loop immediately,
skips all remaining iterations,
execution continues after
the loop block,
use when target is found**



CONTINUE

**skips only the current iteration,
returns to condition check,
remaining iterations still run,
use when filtering values**

Choosing between `break` and `continue` depends on intent: use `break` when you are *done* with the loop entirely, and `continue` when you want to *skip one round* but keep looping.

7.4 Input Validation with while

One of the most practical uses of the `while` loop is **input validation** – forcing the user to re-enter data until a valid value is provided. The loop continues asking as long as the input falls outside the acceptable range.



7.4 Input Validation – Code Example

File: 07validate.py – Accept a score between 0 and 100; reject and re-prompt if out of range.

Source Code

```
score = int(input("Enter score (0-100): "))

while score < 0 or score > 100:
    print("Invalid score. Please try again.")
    score = int(input("Enter score (0-100): "))

print(f"Score {score} accepted.")
```

Sample Run

```
Enter score (0-100): 150
Invalid score. Please try again.
Enter score (0-100): 85
Score 85 accepted.
```

Design Notes

The first `input()` call is placed **before** the loop so the initial value can be tested. The same prompt is repeated inside the loop. This "read-then-validate" pattern is the standard approach for input validation in Python.

7.4 Why while Is Perfect for Validation

Unknown Number of Attempts

We cannot predict how many times a user will enter an invalid value. A `for` loop requires a fixed count; `while` adapts to however many attempts are needed.

Condition-Driven Exit

The loop exits precisely when the condition is satisfied – no wasted iterations, no premature exit. The exit condition directly encodes the validity rule.

Reusable Pattern

The same validate-and-reprompt pattern applies to any type of input: numeric ranges, string formats, menu choices, file names, and more.



The `while` loop is especially powerful for **numerical computation** – iteratively refining a value until it converges to a solution. Newton's method for finding square roots is a classic example: starting from an initial guess, the algorithm repeatedly improves the estimate until the error is smaller than a specified tolerance.

7.5 Newton's Method – The Mathematics

The Algorithm

To find $\sqrt{S}$, start with an initial guess $x_0 = S/2$ and apply the update rule:

$$x_{n+1} = \frac{x_n + S/x_n}{2}$$

Repeat until $|x^2 - S| < \varepsilon$ where ε is the desired tolerance (e.g., 0.0001).

Why It Converges

Each iteration averages the current guess with S/x . If x is too large, S/x is too small, and their average is closer to the true root. The method converges **quadratically** – the number of correct decimal places roughly doubles each iteration.

7.5 Newton's Method – Code

File: 07newton.py – Compute the square root of 25 using Newton's iterative method.

Source Code

```
S = 25
x = S / 2 # initial guess

while abs(x * x - S) > 0.0001:
    x = (x + S / x) / 2

print(f"Square root of {S} ≈ {x:.4f}")
```

Output

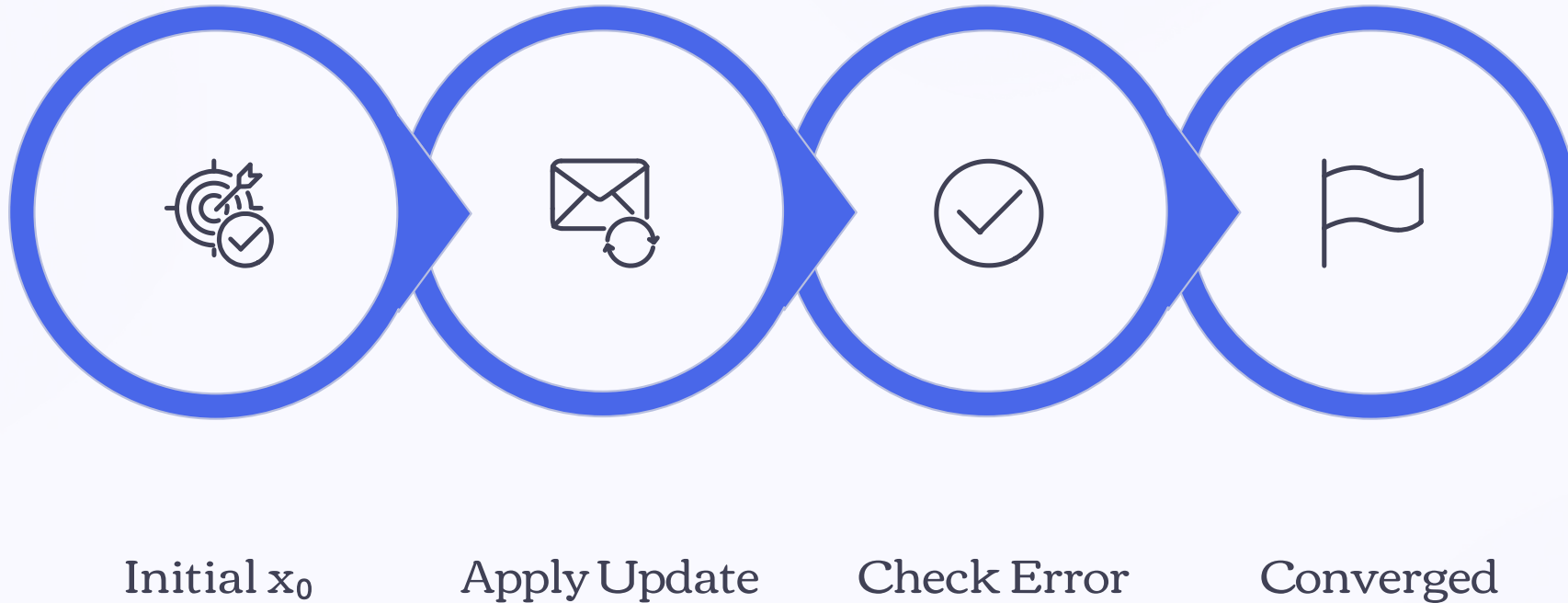
Square root of 25 ≈ 5.0000

Iteration Trace

Starting with $x = 12.5$, the loop refines the estimate: $7.25 \rightarrow 5.349 \rightarrow 5.011 \rightarrow 5.0000$. The loop exits as soon as $|x^2 - 25| < 0.0001$.

This demonstrates the power of `while` for convergence-based algorithms where the number of iterations is not known in advance.

7.5 Newton's Method – Convergence Visualization



The loop body applies the update formula and the condition checks the error. Because Newton's method converges very rapidly, only a handful of iterations are typically needed even for high precision. This makes `while` the natural choice – a `for` loop would require guessing the iteration count in advance.

7.6 Applied Example: Digit Sum of an Integer

The `while` loop combined with the `%` (modulo) and `//` (integer division) operators provides an elegant way to process individual digits of an integer without converting it to a string.

The Idea

`n % 10` extracts the last (units) digit of `n`. `n // 10` removes the last digit, shifting all remaining digits one place right. Repeating this until `n = 0` processes every digit from right to left.

Mathematical Basis

For any integer n , the digit sum is:

$$\text{digitsum}(n) = \sum_k d_k \quad \text{where } d_k = \lfloor n/10^k \rfloor \bmod 10$$

The loop implements this extraction iteratively without needing to know the number of digits in advance.

7.6 Digit Sum – Code and Output

File: 07digitsum.py – Sum all digits of a positive integer entered by the user.

Source Code

```
n = int(input("Enter a positive integer: "))
total = 0

while n > 0:
    total += n % 10 # extract units digit
    n = n // 10    # remove units digit

print(f"Sum of all digits = {total}")
```

Sample Run

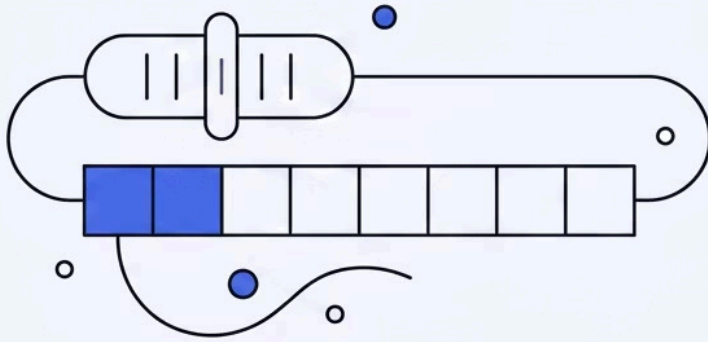
```
Enter a positive integer: 4729
Sum of all digits = 22
```

Trace for 4729

Round 1: total += 9, n = 472 | Round 2: total += 2, n = 47 | Round 3:
total += 7, n = 4 | Round 4: total += 4, n = 0 → loop exits. Total =
9+2+7+4 = **22**.

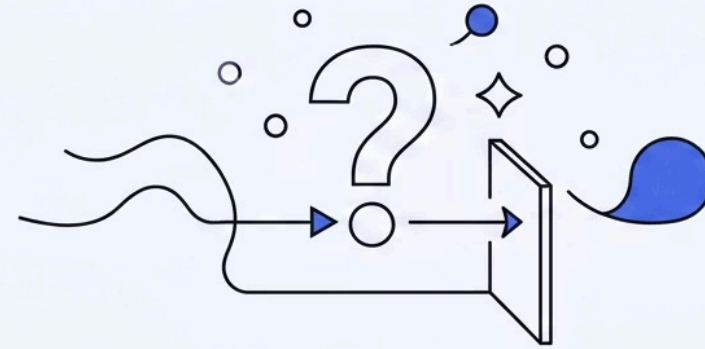
for vs. while – When to Use Each

FOR LOOP



- **Known iterations**
- **Iterates over sequence**
- **Counting problems**
- **Fixed repetition**

WHILE LOOP



- **Unknown iterations**
- **Condition-driven**
- **Input validation**
- **Searching until found**

The choice between `for` and `while` is a design decision. When the iteration count is determined before the loop starts, `for` is cleaner and safer. When the exit depends on a runtime condition, `while` is the appropriate and more expressive choice.

Common while Loop Patterns



Counter-Controlled

Initialize a counter, check it in the condition, increment inside the loop. Equivalent to a `for` loop but more explicit. Use when you need more control over the counter variable.



Flag-Controlled

Use a boolean variable (flag) as the condition. Set it to `False` inside the loop when the exit condition is met. Useful when the exit logic is complex.



Sentinel-Controlled

Loop until a special "sentinel" value is entered (e.g., 0 or -1). The sentinel signals the end of input. Common in data-entry programs.



Convergence-Controlled

Loop until the difference between successive values falls below a tolerance ϵ . The standard pattern for iterative numerical methods like Newton's method.



7.7 Chapter Summary

The `while` loop repeats as long as its condition remains true, making it ideal for situations where the number of iterations is not known in advance. Here are the key takeaways from Chapter 7:

7.7 Key Takeaways

1

Condition-First Execution

The `while` loop checks its condition before every iteration. If the condition is false from the start, the loop body never executes.

2

Prevent Infinite Loops

Always ensure the variable(s) in the condition are modified inside the loop body so the condition can eventually become false.

3

`break` Exits Immediately

`break` terminates the loop at once, regardless of the condition. Use it when a search target is found or an exit condition is detected mid-loop.

4

`continue` Skips One Round

`continue` skips the rest of the current iteration and returns to the condition check. Use it to filter out unwanted values without exiting the loop.

✓ The `while` loop is especially valuable for **input validation** and **numerical convergence** – two fundamental patterns in mathematical programming.

End-of-Chapter Exercises

The following five exercises reinforce the concepts from Chapter 7. Each exercise requires writing a complete Python program using `while` loops.

1 Countdown

Write a program that displays the numbers 10 down to 1 using a `while` loop.

2 Sum Exceeding 1000

Write a program that sums 1, 2, 3, ... until the total exceeds 1000, then displays which number was reached last.

3 Running Total Until Zero

Write a program that repeatedly accepts numbers from the user until 0 is entered, then displays the sum of all entered numbers.

4 Count Digits

Write a program that counts the number of digits in a positive integer entered by the user.

5 Prime Number Check

Write a program that checks whether an input number is prime, using `while` and `break`.

Exercise 1: Countdown from 10 to 1

Write a `while` loop that prints the integers from 10 down to 1. This is the reverse of the standard counting-up pattern and requires careful condition and update design.

Approach

Initialize `i = 10`. The condition is `i >= 1`. Inside the loop, print `i` then decrement with `i -= 1`. The loop exits when `i` drops to 0.

Expected Output

```
10
9
8
7
6
5
4
3
2
1
```

Exercise 2 Solution: Sum Exceeding 1000

Sum the integers 1, 2, 3, ... until the running total exceeds 1000. Display the last number added and the final total.

Source Code

```
total = 0
i = 0

while total <= 1000:
    i += 1
    total += i

print(f"Reached number {i}, sum = {total}")
```

Output

Reached number 45, sum = 1035

Explanation

The loop continues as long as `total <= 1000`. Each iteration increments `i` and adds it to `total`. When `i = 45`, the total becomes 1035 which exceeds 1000, so the condition becomes false and the loop exits. The sum of 1 through 45 is $\frac{45 \times 46}{2} = 1035$.

Exercise 3 Solution: Running Total Until Zero

Repeatedly accept integers from the user. When the user enters 0, stop and display the sum of all previously entered numbers.

Source Code

```
total = 0
x = int(input("Enter number (0 to quit): "))

while x != 0:
    total += x
    x = int(input("Enter number (0 to quit): "))

print(f"Sum = {total}")
```

Sample Run

```
Enter number (0 to quit): 10
Enter number (0 to quit): 20
Enter number (0 to quit): 0
Sum = 30
```

Pattern Note

This is the **sentinel-controlled** loop pattern. The value 0 acts as the sentinel – a special signal that terminates input. The first `input()` before the loop primes the condition; the same prompt inside the loop continues reading. The sentinel value itself (0) is *not* added to the total.

Exercise 4 Solution: Count the Digits

Count how many digits a positive integer contains by repeatedly dividing by 10 until the number reaches 0.

Source Code

```
n = int(input("Enter a positive integer: "))
count = 0

while n > 0:
    count += 1
    n = n // 10

print(f"Number of digits: {count}")
```

Sample Run

```
Enter a positive integer: 50321
Number of digits: 5
```

Trace for 50321

50321 → 5032 → 503 → 50 → 5 → 0 (loop exits). Counter increments once per iteration: **5 digits**. This uses the same digit-stripping technique as the digit sum example in Section 7.6, but counts iterations instead of accumulating a sum.

Exercise 5: Prime Number Check

Check whether a given positive integer is prime using a `while` loop and `break`. A number is prime if it has no divisors other than 1 and itself.

Algorithm

Test divisors from 2 up to $\lfloor \sqrt{n} \rfloor$. If any divisor divides `n` evenly, the number is not prime – use `break` to exit early. If the loop completes without finding a divisor, the number is prime.

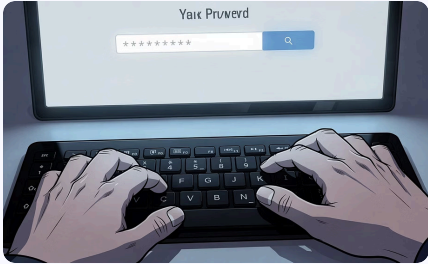
Approach Outline

```
n = int(input("Enter a positive integer: "))
d = 2
is_prime = True

while d * d <= n:
    if n % d == 0:
        is_prime = False
        break
    d += 1

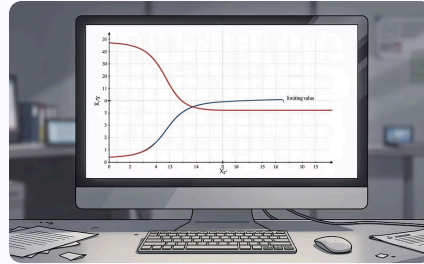
if is_prime and n > 1:
    print(f"{n} is prime")
else:
    print(f"{n} is not prime")
```

Putting It All Together – while Loop Applications



Input Validation

Enforce valid data entry by looping until the user provides an acceptable value. Prevents downstream errors from bad input.



Numerical Convergence

Iterate a formula until the result stabilizes within a tolerance. Powers algorithms like Newton's method, gradient descent, and fixed-point iteration.



Search Problems

Scan through values until a target condition is met, then exit with `break`. Efficient because the loop stops as soon as the answer is found.

The while True Pattern

Using `while True` creates an intentionally infinite loop that relies on `break` for all exits. This pattern is useful when the exit condition is complex, occurs mid-loop, or when multiple exit points are needed.

Structure

```
while True:
    # do work
    if exit_condition_1:
        break
    # do more work
    if exit_condition_2:
        break
```

When to Use

- The exit condition is naturally checked in the middle of the loop body
- Multiple different exit conditions exist
- The loop must execute at least once before checking (do-while equivalent)
- The exit logic is clearer when written as an `if/break` than as a `while` condition

 Always ensure at least one `break` is reachable – otherwise the loop truly runs forever.

Nested while Loops

A `while` loop can contain another `while` loop inside it. The inner loop completes all its iterations for each single iteration of the outer loop. `break` and `continue` affect only the **innermost** loop they appear in.

Example: Multiplication Table

```
i = 1
while i <= 3:
    j = 1
    while j <= 3:
        print(f"{i}×{j}={i*j}", end=" ")
        j += 1
    print()
    i += 1
```

Output

```
1×1=1 1×2=2 1×3=3
2×1=2 2×2=4 2×3=6
3×1=3 3×2=6 3×3=9
```

Key Point

Each variable (`i` and `j`) must be managed independently. Forgetting to reset the inner counter (`j = 1`) before the inner loop is a common bug in nested loop programs.

Debugging while Loops



Loop Never Executes

The condition is false before the loop starts. Check initial values of variables used in the condition.



Infinite Loop

The condition never becomes false. Verify that the loop variable is updated correctly inside the body. Use Ctrl+C to stop.



Off-by-One Error

The loop runs one too many or too few times. Check whether the condition uses `<` vs `<=` and whether the update happens before or after the work.



Add Print Statements

Insert `print()` calls inside the loop to trace variable values each iteration. This is the simplest and most effective debugging technique.

while Loop vs. for Loop – Complete Comparison

Aspect	for Loop	while Loop
Iteration count	Known before loop starts	Unknown; determined at runtime
Condition check	Implicit (exhausts sequence)	Explicit boolean expression
Counter management	Automatic	Manual (programmer's responsibility)
Infinite loop risk	Low (finite sequence)	Higher (must update condition variable)
Best use cases	Iterating lists, ranges, sequences	Validation, convergence, search
break / continue	Supported	Supported

Summary of All Code Examples

1

`O7while.py`

Basic `while` loop counting from 1 to 5. Demonstrates the fundamental structure: initialize, condition, body, update.

2

`O7break.py`

Find the smallest integer whose square exceeds 100 using `while True` and `break`. Answer: 11.

3

`O7continue.py`

Print only odd numbers 1 - 9 by using `continue` to skip even values inside the loop.

4

`O7validate.py`

Input validation loop: re-prompt until a score in `[0, 100]` is entered. Classic sentinel/validation pattern.

5

`O7newton.py`

Newton's method for $\sqrt{25}$: iterate until `|x2-S| < 0.0001`. Result: 5.0000.

6

`O7digitsum.py`

Sum digits of 4729 using `% 10` and `// 10`. Result: 22.

Exercise Solutions at a Glance

45

Exercise 2 Answer

The sum $1+2+\dots+45 = 1035$ is the first to exceed 1000.

30

Exercise 3 Sample

Entering 10, 20, then 0 gives a running total of 30.

5

Exercise 4 Answer

The integer 50321 has 5 digits, counted by repeated `// 10`.

 Exercise 1 (countdown) and Exercise 5 (prime check) are left as student practice. The prime check solution outline is provided in slide 31.



CLO 3 Alignment

The content of Chapter 7 directly supports **Course Learning Outcome 3 (CLO 3)**: the ability to design and implement programs that use conditional repetition structures to solve mathematical and computational problems. Students who master this chapter will be equipped to handle open-ended iteration – a fundamental skill in scientific computing, data processing, and algorithm design.

Chapter 7 Complete

Core Concept

`while` loops repeat as long as a condition is true – ideal when the iteration count is unknown.

Safety Rule

Always update the condition variable inside the loop. Use Ctrl+C to stop an infinite loop during testing.

Control Flow

`break` exits immediately; `continue` skips to the next iteration. Both work in `while` and `for` loops.

Key Applications

Input validation, numerical convergence (Newton's method), digit processing, and sentinel-controlled input.

✅ Next: Chapter 8 – Functions. Prepare by reviewing how to break problems into reusable sub-tasks.